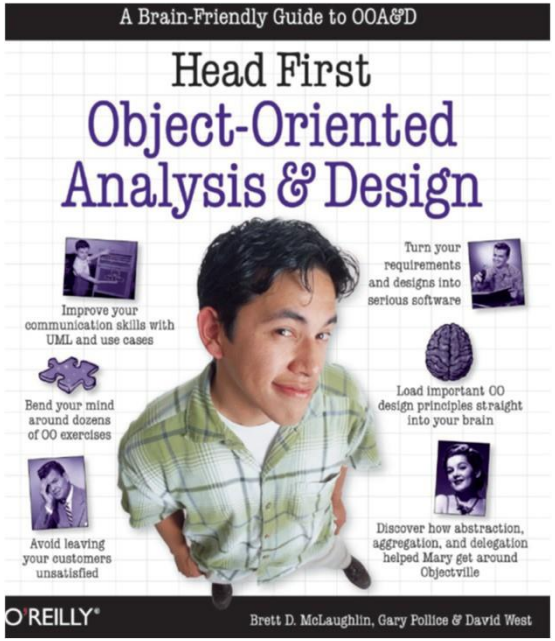
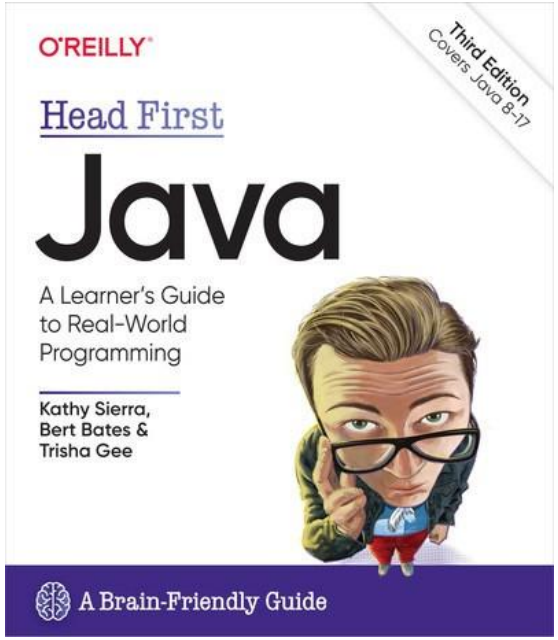
Software Development for Mobile Devices

Lecture4- part 2

Dr.Marwa Elseddik
Lecturer, Department of Robotics and Intelligent Machines
2025-2026


A Brain-Friendly Guide to OOA&D
Head First
Object-Oriented
Analysis & Design
Improve your communication skills with UML and use cases
Turn your requirements and designs into serious software
Bend your mind around dozens of OO exercises
Load important OO design principles straight into your brain
Avoid leaving your customers unsatisfied
Discover how abstraction, aggregation, and delegation helped Mary get around Objectville
O'REILLY
Brett D. McLaughlin, Gary Pollice & David West


O'REILLY
Third Edition
Covers Java 8-17
Head First
Java
A Learner's Guide to Real-World Programming
Kathy Sierra, Bert Bates & Trisha Gee
A Brain-Friendly Guide

References

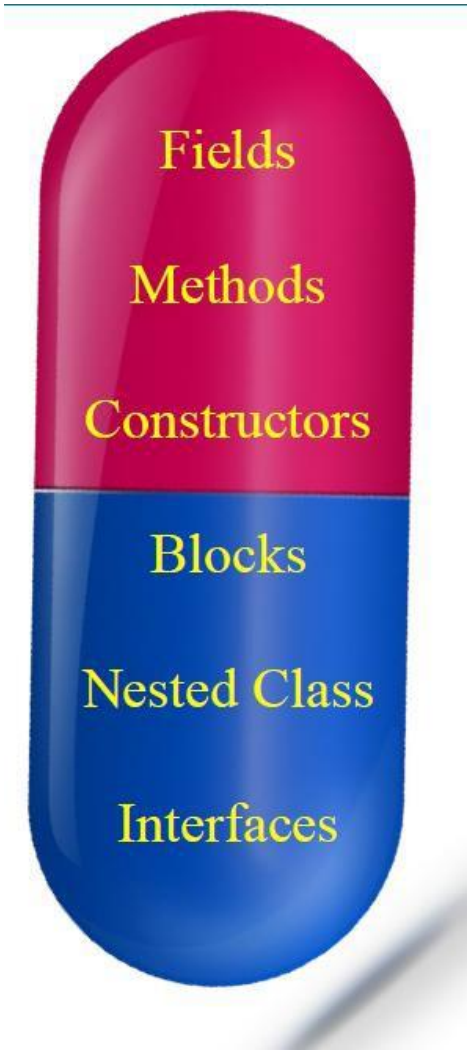
TO BE CONTINUED …

Encapsulation

Encapsulation (1)

- **Encapsulation** is when you **hide** the implementation of a class in such a way that it is easy to use and easy to change.

- It makes the class act as a **black box** that provides a service to its users but does not open up the code so someone **can change it or use it the wrong way.**



A Java class

Encapsulation (2)

Formally, **encapsulation** is defined as:

OOP
Interview
questions!

The process of enclosing programming elements inside larger, more abstract entities. Also known as information hiding or separation of concerns.

Encapsulation (3)

- So, what's the opposite of Encapsulation?
  - Exposing our data!

- Exposed means reachable with the dot operator, as in:

theCat.height = 27;

```
theCat.height = 0;
```

Oh my goodness! We can't let this happen!

Suggested fix: We need to build setter methods for all the instance variables and force other code to call the setters rather than access the data directly.

Encapsulation
How?

```
theCat.height = 0;
```

Oh my goodness! We can't let this happen!

- **Hide the data:** Using **public** and **private** access modifiers.
  - Mark instance variables **private**.
  - Mark ***getters*** and ***setters*** **public**.

By forcing everybody to call a setter method, we can protect the cat from unacceptable size changes.

```
public void setHeight(int ht) {
    if (ht > 9) {
        height = ht;
    }
}
```

We put in checks to guarantee a minimum cat height.

Important notes - Encapsulation

- Encapsulation gives you control over **who changes the data in your class and how.**

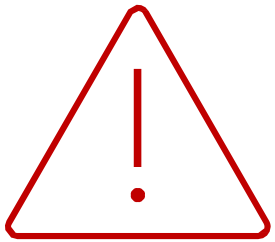
- Make an **instance variab**le *private* so it **can't** be changed by accessing the variable directly.

- Create a public **mutator method**, e.g., a setter, to control how other code interacts with your data. For example, you can add a validation code inside a setter to make sure the value isn't changed to something invalid.

Encapsulation or Not?

```
public class Airplane {

  public int speed;

  public Airplane() {
  }

  public void setSpeed(int speed) {
    this.speed = speed;
  }

  public int getSpeed() {
    return speed;
  }
}
```

We made the speed variable public, instead of private, and now all parts of your app can access speed directly.



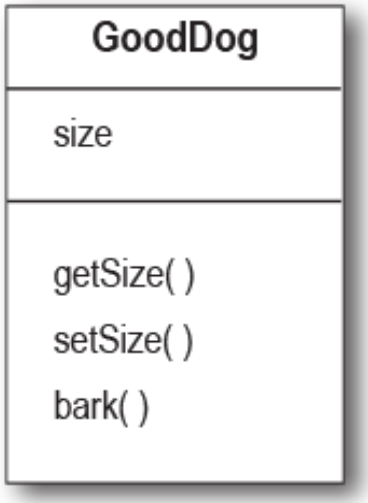
Anyone can set the speed directly!

Example 1: Encapsulating a GoodDog (1)

Make the instance variable private.

Make the getter and setter methods public.

```
class GoodDog {
    private int size;

    public int getSize() {
        return size;
    }

    public void setSize(int s) {
        size = s;
    }

    void bark() {
        if (size > 60) {
            System.out.println("Wooof! Wooof!");
        } else if (size > 14) {
            System.out.println("Ruff! Ruff!");
        } else {
            System.out.println("Yip! Yip!");
        }
    }
}
```



Even though the methods don't really add new functionality, the nice thing is that you can change your mind later. You can come back and make a method safer, faster, better.

Example 1: Encapsulating a GoodDog (2)

```
class GoodDog {
   private int size;

   public int getSize() {
      return size;
   }

   public void setSize(int s) {
      size = s;
   }

   void bark() {
      if (size > 60) {
         System.out.println("Wooof! Wooof!");
      } else if (size > 14) {
         System.out.println("Ruff! Ruff!");
      } else {
         System.out.println("Yip! Yip!");
      }
   }
}
```

```
class GoodDogTestDrive {
   public static void main(String[] args) {
      GoodDog one = new GoodDog();
      one.setSize(70);

      GoodDog two = new GoodDog();
      two.setSize(8); \\method parameter

      System.out.println("Dog one: " + one.getSize());
// return size
      System.out.println("Dog two: " + two.getSize());

      one.bark();
      two.bark();
   }
}
```

Scope Rule in Java

Scope Rule

```
class Scope {
   public static void main(String args[]) {
     int x; // known to all code within main
     x = 10;
      if(x == 10) {              // start new scope
         int y = 20;             // known only to this block
         System.out.println("x and y: " + x + " " + y);
         x = y * 2;     // x and y both are known here.
      }
      y = 100;  // Error! y is now no more known here
      System.out.println("x is " + x); // x is still known
      here.
   }
}
```

Instance variable versus Class variable

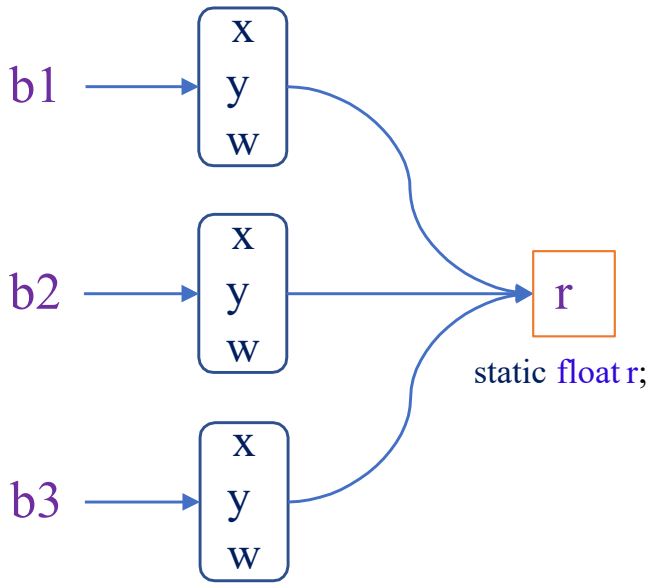
Two instances of class **Box**

Two instances of class **Circle**

- In class **`Box`** and class **`Circle`**, we declared three variables (x, y, w) and (x, y, r), respectively.
- Such a variable is called **instance variable**.
- They are so called because each instance of the class, say, **`Circle`**, has its own copy.

Instance Variable versus Class Variable

- Java does not allow global variables.
- Every variable in Java must be declared inside a class.
- The keyword **`static`** is used to make a variable just like global variable.
- A variable declared with **`static`** keyword is called **class variable.**
- It acts like a global variable, that is, there is only <u>one copy</u> of the variable associated with the class.



Three instances of class **Box**

That is one copy of the variable regardless of the number of instances of the class.

Static Variable: An Example

```
public class Circle{
   static int circlecount = 0; // class variable (shared among all objects)
   public double x, y, r; // instance variables

   //constructor with parameters
   public Circle(double x, double y, double r){
      this.x = x;
      this.y = y;
      this.r = r;
      circlecount++; // Increment count for each new object
   }
   public static void main(String args[ ]){
      Circle c1 = new Circle(2.0, 3.0, 5.0); // Creates first circle
      Circle c2 = new Circle(0.0, 0.0, 4.0); // Creates second circle
      Circle c3 = new Circle(1.0, 1.0, 3.0); // Creates third circle
      // Demonstrating that circleCount is shared among all instances
      System.out.println("c1 count" + Circle.circlecount );
      System.out.println("c2 count" + Circle.circlecount );
      System.out.println("c3 count" + Circle.circlecount );
      System.out.println("Total Circles Created: " + Circle.circlecount);
   }
}
```

Referring to the Current Instance

In an instance method or constructor, this refers to the current object of the class. It's often used to distinguish between instance variables and method parameters when they have the same name.

```
class Car {
    String model; // Instance variable
    int year;     // Instance variable
    // Constructor
    Car(String model, int year) {
        this.model = model; // 'this.model' refers to the instance variable, while 'model' is the constructor parameter
        this.year = year;   // 'this.year' refers to the instance variable, while 'year' is the constructor parameter
    }

    public void displayDetails() {
        System.out.println("Model: " + this.model); // Referring to the current object's model
        System.out.println("Year: " + this.year);   // Referring to the current object's year
    }
}
```

```
Car c1 = new Car("Toyota", 2020);
Car c2 = new Car("BMW", 2022);
```

```
c1.displayDetails();  // يعرض تفاصيل Toyota
c2.displayDetails();  // يعرض تفاصيل BMW
```

Scope Rule: Another Example

The **scope rule** refers to the region of the program where a particular variable, method, or class is accessible or visible.

```
class Box{
   float x = 10.0f;
   float y = 20.0f;
   float w = 15.0f;

   float area(){
     return(2*(x*y + x*w + y*w));
   }
   }
```

```
class Circle{
   float x = 0.0f;
   float y = 0.0f;
   float r = 5.0f;

   float area(){
     return(((22/7)*r*r));}

}
```

Error: non-static variable x cannot be referenced from a static context

you cannot access instance variables (non-static fields) directly within a static method unless you use an object of the class

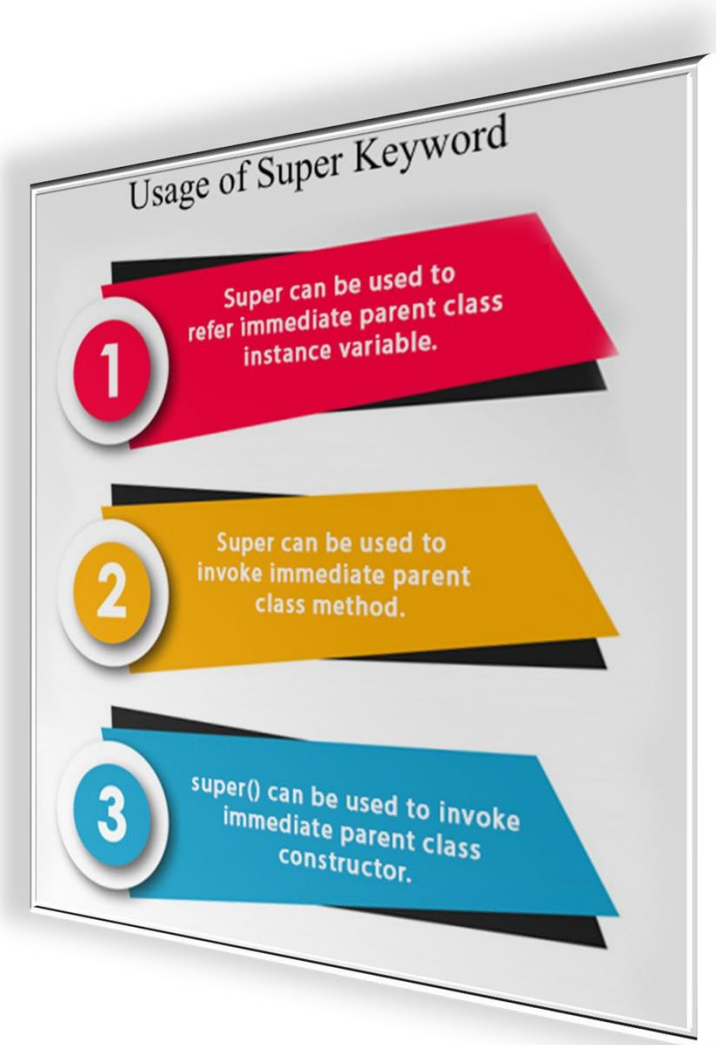
```
class GeoClass{
   float x = 50;
   float y = 60;

   public static void main(String args[]) {
        Box b = new Box();
        Circle c = new Circle();
        System.out.println("GeoClass Data: x = " + x);
        System.out.println("Box Data: x = " + b.x);
        System.out.println("Box Area: " + b.area());
        System.out.println("Circle Data: x = " + c.x);
        System.out.println("Circle Area: " + c.area());
   }
}
```

`super` Keyword

`super` Keyword concept in Java

The **`super`** keyword in Java is a reference variable which is used to refer immediate parent class members.

Whenever you create an instance of a sub class, an instance of its parent class is created implicitly, which is referred by **`super`** keyword.



super : Refer ing parent class instance variable

```
class Animal{
    String color="white";
}

class Dog extends Animal{
String color ="black";
  void printColor( ){
      System.out.println(color);
      System.out.println(super.color);
  }
}
class TestSuper1{
    public static void main(String args[]){ Dog
    d = new Dog();
    d.printColor( );
  }
}
```

Animal and **Dog** both classes have a common property color. If you print the color property, it will print the color of the current class by default. To access the parent property, you should use `super` keyword.

```
black
white
```

super: Invoking parent class method

```
class Animal{
    void eat( ) {
        System.out.println("eating...");
    }
}
class Dog extends Animal{
    void eat( ){
        System.out.println("eating bread...");
    }
    void bark( ){
        System.out.println("barking...");
    }
    void work( ){
        super.eat( );
        bark( );
        eat( );
    }
    }
class TestSuper2{
    public static void main(String args[ ]){
        Dog d = new Dog();
        d.work();
    }
}
```

`Animal` and `Dog` both the classes have `eat()` method. If you call `eat()` method from `Dog` class, it will call the `eat()` method of `Dog` class by default because priority is given to local.

To call the parent class method, you need to use `super` keyword.

```
eating...
barking...
Eating bread...
```

super: Invoking parent class constructor

```
class Animal{
  Animal( ){
          System.out.println("animal is created");
  }
}

class Dog extends Animal{
     Dog( ){
          super( );
          System.out.println("dog is created");
     }
}

class TestSuper3{
    public static void main(String args[ ]){
        Dog d = new Dog( );
    }
}
```

The `super` keyword can also be used to invoke the overloaded parent class constructor, if arguments are there, then they should be specified accordingly.

```
animal is created
dog is created
```

Thanks